KỊCH BẢN NÓI KHI BẢO VỆ

Sinh tự động từ `scripts/noi-dung-slide.py` bằng `scripts/tao-kich-ban.py`. Sửa ghi chú ở đó rồi chạy lại **cả hai** script — đừng sửa thẳng vào tệp này, lần dựng sau sẽ mất.

Phần lời nói ở đây **giống hệt** phần ghi chú người trình bày trong tệp PowerPoint. Mở chế độ người trình bày là thấy lời nói ngay dưới slide; tệp này để in ra cầm theo hoặc đọc lại khi tập.

Hai bản, một bộ slide

Bộ slide có 28 slide. Bạn **không nói hết** khi bảo vệ.

Bản	Slide	Số từ	Thời lượng
Đường ngắn — nói khi bảo vệ	18	1806	**~12 phút**
Bản đầy đủ — dùng cho video	28	2784	~19 phút

Khi bảo vệ đi theo *đường ngắn*: hội đồng cho 10–20 phút *kể cả hỏi đáp*, nên phần trình bày phải gọn trong khoảng 12 phút để còn chỗ cho phần hỏi. Những slide đánh dấu `[bản đầy đủ]` thì **bấm qua, không dừng lại nói**.

Khi quay video nộp LMS thì nói hết cả 28 slide, vì video không bị bấm giờ gắt như buổi bảo vệ và nói đủ thì người chấm thấy được chiều sâu của đồ án.

Nguyên tắc chọn đường ngắn: bỏ những slide **mô tả** (giao diện trông thế nào, có bao nhiêu màn hình), giữ những slide trả lời câu hỏi **"vì sao"** — vì đó là chỗ hội đồng chấm điểm.

> **Cách tập:** đọc to một lượt và bấm giờ thật. Đọc thầm bao giờ cũng nhanh hơn nói thật khoảng một phần ba. Mỗi slide dưới đây đều ghi sẵn số từ và số giây để bạn biết mình đang vượt ở đâu.

Slide 1 — Bìa

(44 từ · ~18 giây)

Nói:

Em chào thầy cô. Em là Nguyễn Hồng Quân, lớp DX23TT11. Hôm nay em xin trình bày đồ án thực tập chuyên ngành: Xây dựng website giới thiệu và đặt phòng homestay cho Homestay TVH, dưới sự hướng dẫn của thầy Nguyễn Nhứt Lam.

Slide 2 — Nội dung trình bày

Trên slide: Bài toán và mục tiêu · Vấn đề kỹ thuật then chốt · Thiết kế và cài đặt · Kết quả kiểm chứng · Giới hạn và hướng phát triển

(61 từ · ~24 giây)

Nói:

Phần trình bày của em gồm năm mục. Em xin phép dành nhiều thời gian nhất cho mục thứ hai — vấn đề kỹ thuật then chốt — vì đó là chỗ em đầu tư công sức nhiều nhất và cũng là chỗ em nghĩ có giá trị nhất của đề tài. Ba mục còn lại em sẽ đi nhanh hơn.

Slide 3 — Bài toán: homestay quản lý phòng bằng trí nhớ

Trên slide: Đặt phòng đến từ nhiều kênh tin nhắn khác nhau · Không có nơi nào cho biết một ngày còn bao nhiêu phòng · Sàn trung gian thu 15–20% mỗi đơn

Hình: `hinh-1-2-quy-trinh-thu-cong.png`

(118 từ · ~47 giây)

Nói:

Du lịch homestay ở Trà Vinh đang phát triển, nhưng phần lớn cơ sở là hộ gia đình tự vận hành. Chủ nhà nhận đặt phòng qua nhiều kênh tin nhắn khác nhau, mỗi kênh là một danh sách riêng nằm trong trí nhớ. Lưu đồ trên slide là quy trình thủ công hiện nay, và ba ô tô đỏ là ba chỗ hỏng: tra cứu lịch bằng trí nhớ, không có bản ghi tập trung nên mất sổ là mất hết, và nguy hiểm nhất là hai khách nhắn gần như cùng lúc thì đều được trả lời là còn phòng. Còn nếu đưa lên sàn trung gian thì mất 15 đến 20 phần trăm mỗi đơn.

Slide 4 — Vì sao không đưa lên sàn có sẵn `[bản đầy đủ]`

Trên slide: Sàn thu 15–20% mỗi đơn — với homestay nhỏ là phần lớn lợi nhuận · Chủ nhà không sở hữu dữ liệu khách của chính mình · Vẫn phải tự giữ một lịch riêng cho khách đặt trực tiếp

(114 từ · ~46 giây)

Nói:

> Một câu hỏi hợp lý là vì sao không đưa luôn lên Booking hay Agoda cho nhanh. Em có khảo sát và thấy ba vấn đề. Thứ nhất là hoa hồng 15 đến 20 phần trăm mỗi đơn, với một homestay nhỏ thì đó là phần lớn lợi nhuận. Thứ hai, chủ nhà không sở hữu dữ liệu khách của chính mình, nên không xây được tệp khách quen. Thứ ba, và đây mới là điều quan trọng: kể cả bán trên sàn thì chủ nhà vẫn phải giữ một lịch riêng cho khách đặt trực tiếp qua tin nhắn — tức là bài toán trùng lịch vẫn còn nguyên, chỉ là chuyển chỗ.

Slide 5 — Bài toán cốt lõi: không được bán trùng phòng

Trên slide: Hai khách liên hệ gần như cùng lúc · Chủ nhà trả lời còn phòng cho cả hai · Sai sót chỉ lộ ra vào ngày nhận phòng

(95 từ · ~38 giây)

Nói:

> Em muốn dừng lại ở rủi ro thứ ba, vì nó mới là bài toán kỹ thuật thật sự. Đây không phải lỗi bất cẩn của chủ nhà. Đây là hệ quả tất yếu của việc quản lý lịch phòng bằng trí nhớ con người: giữa lúc nhìn vào sổ và lúc ghi vào sổ luôn có một khoảng trống thời gian, và trong khoảng đó người thứ hai chen vào được. Hậu quả thì chỉ lộ ra vào ngày nhận phòng, khi khách đã tới nơi — lúc đó không còn cách nào sửa.

Slide 6 — Quy ước nửa mở cho khoảng ngày

Trên slide: Trả phòng 10/03 và nhận phòng 10/03 KHÔNG chồng lấn · Viết là [nhận, trả) — chứa ngày đầu, không chứa ngày cuối · Sai quy ước này là mất một đêm bán được của mỗi phòng

Hình: `hinh-1-1-khoang-nua-mo.png`

(109 từ · ~44 giây)

Nói:

> Trước khi nói chuyện chống đặt trùng, phải thống nhất với nhau thế nào là trùng. Hai thanh trên hình chạm nhau tại ngày mùng mười tháng ba, nhưng chúng không chồng lấn: khách thứ nhất trả phòng buổi sáng, khách thứ hai nhận phòng buổi chiều, cùng một ngày mà vẫn bán được cho cả hai. Em dùng quy ước nửa mở, viết là ngoặc vuông mở ngoặc tròn đóng, nghĩa là chứa ngày nhận nhưng không chứa ngày trả. Nếu làm sai quy ước này thì mỗi phòng mất một đêm bán được ở mỗi lượt khách — một lỗi nhỏ trên giấy nhưng rất tốn tiền.

Slide 7 — Mục tiêu và phạm vi

Trên slide: Ba khối: trang khách, khu quản trị, giao diện lập trình ứng dụng · Đặt cọc qua mã QR, webhook tự xác nhận · Không có nhánh nào để tiền của khách biến mất im lặng · Một lệnh là đủ để chạy toàn hệ thống kèm dữ liệu mẫu

(82 từ · ~33 giây)

Nói:

> Từ bài toán đó, đồ án đặt ra tám mục đích cụ thể, và điều em cố giữ là mỗi mục đích phải đi kèm một tiêu chí nghiệm thu đo được bằng lệnh, chứ không phải một lời hứa. Hệ thống gồm ba khối: trang bán hàng cho khách, khu quản trị cho chủ homestay, và giao diện lập trình ứng dụng nối hai khối đó. Khách đặt cọc bằng mã QR và hệ thống tự xác nhận khi tiền về.

Slide 8 — Phân tích chức năng: 11 use case, 4 tác nhân

Hình: `hinh-3-1-use-case.png`

(123 từ · ~49 giây)

Nói:

> Về phân tích chức năng, hệ thống có mười một use case và bốn tác nhân. Cột trái là nhóm chức năng của khách: tìm phòng, đặt phòng, thanh toán cọc, tra cứu và huỷ đơn, xem lại đơn đã đặt, viết đánh giá. Cột phải là của quản trị viên: đối soát

thanh toán, quản lý đơn, báo cáo doanh thu, đặt ngày khả dụng, soạn nội dung trang chủ. Em xin lưu ý quan hệ kế thừa ở góc trái: khách có tài khoản làm được mọi thao tác của khách vãng lai và chỉ thêm một chức năng là xem lại toàn bộ đơn đã đặt — nên hệ thống không bắt ai phải đăng ký mới đặt được phòng.

Slide 9 — Công nghệ sử dụng

Trên slide: Máy chủ: Java 21, Spring Boot 3.5 · Giao diện: Angular 21, Tailwind CSS v4 · Cơ sở dữ liệu: PostgreSQL 16 — chọn vì một lý do rất cụ thể · Đóng gói: Docker Compose, bốn dịch vụ

(80 từ · ~32 giây)

Nói:

Về công nghệ, em dùng Java 21 với Spring Boot cho tầng máy chủ, Angular 21 cho tầng giao diện, và đóng gói bằng Docker Compose. Em xin nhấn mạnh dòng thứ ba: em chọn PostgreSQL không phải vì nó mạnh hơn MySQL một cách chung chung, mà vì một tính năng rất cụ thể mà MySQL không có. Hai slide nữa em sẽ nói rõ tính năng đó là gì và vì sao nó quyết định cả đề tài.

Slide 10 — Kiến trúc tổng thể `[bản đầy đủ]`

Hình: `hinh-2-1-kien-truc-tong-the.png`

(93 từ · ~37 giây)

Nói:

Hệ thống chạy bốn dịch vụ. Máy chủ web là đường vào duy nhất từ bên ngoài; nó phục vụ tệp tĩnh của giao diện và chuyển tiếp các lời gọi vào máy chủ ứng dụng. Cơ sở dữ liệu và hộp thư không mở cổng ra ngoài — chỉ máy chủ ứng dụng gọi tới được. Cách chia này nghĩa là dù ai đó dò được địa chỉ máy, họ cũng không chạm thẳng vào cơ sở dữ liệu, và mọi yêu cầu đều đi qua đúng một cửa có kiểm soát.

Slide 11 — Vì sao kiểm tra ở tầng ứng dụng KHÔNG đủ

Trên slide: Luồng 1 đọc: còn phòng → Luồng 2 đọc: còn phòng · Cả hai cùng ghi → bán trùng · Mức cô lập Read Committed không ngăn được · Tài liệu PostgreSQL khuyên dùng ràng buộc toàn vẹn

Hình: `hinh-2-2-tranh-chap-dat-phong.png`

(143 từ · ~57 giây)

Nói:

> Đây là slide quan trọng nhất của bài. Cách làm tự nhiên nhất khi gặp bài toán này là viết mã kiểm tra: trước khi ghi đơn thì truy vấn xem phòng còn trống không, còn thì mới ghi. Cột bên trái cho thấy vì sao cách đó sai. Hai luồng chạy song song, cả hai cùng đọc và cùng thấy còn phòng, rồi cả hai cùng ghi — kết quả là bán trùng. Khoảng trống giữa lúc đọc và lúc ghi là một khe hở không bịt được bằng logic thông thường. Và điều đáng nói là mức cô lập giao dịch mặc định của PostgreSQL, Read Committed, cũng không ngăn được chuyện này — chính tài liệu của PostgreSQL nêu ví dụ đúng như vậy, và khuyến cáo dùng ràng buộc toàn vẹn thay vì kiểm tra ở tầng ứng dụng.

Slide 12 — Giải pháp: để cơ sở dữ liệu quyết định

Trên slide: EXCLUDE USING gist (room_id WITH =, stay WITH &&) · Cột stay sinh tự động: daterange(nhận, trả, '[)') · Luồng thứ hai nhận lỗi 23P01 và bị bác — ngay trong giao dịch · MySQL không có daterange lẫn ràng buộc loại trừ

(146 từ · ~58 giây)

Nói:

> Nên em không kiểm ở tầng ứng dụng nữa, mà đẩy việc quyết định xuống chính cơ sở dữ liệu. Em khai một ràng buộc loại trừ: hai dòng bị coi là xung đột nếu cùng mã phòng và hai khoảng ngày chồng lấn nhau. Khoảng ngày là một cột sinh tự động kiểu daterange theo quy ước nửa mở em vừa trình bày. Ràng buộc này dùng chỉ số GiST, vì chỉ số B-tree thông thường không so sánh được quan hệ chồng lấn. Khi hai giao dịch cùng ghi, PostgreSQL bác một giao dịch ngay trong động cơ lưu trữ bằng mã lỗi 23P01, không còn khe hở nào để chen vào. Hệ thống bắt đúng mã

lỗi đó rồi tự thử phòng trống tiếp theo trong một giao dịch mới, nên khách thứ hai vẫn đặt được nếu loại phòng còn phòng khác.

Slide 13 — Cơ sở dữ liệu: 21 bảng, 9 migration `[bản đầy đủ]`

Hình: `hinh-3-3-erd-dat-phong.png`

(94 từ · ~38 giây)

Nói:

Lược đồ có 21 bảng nghiệp vụ, dựng bởi chín migration. Hình trên slide là nhóm bảng đặt phòng — nơi đặt ràng buộc em vừa nói. Hai điểm em muốn nêu: thứ nhất, lược đồ do công cụ migration quản lý chứ không để khung ứng dụng tự sinh, và migration là bất biến — đã chạy thì không sửa, chỉ thêm bản mới. Thứ hai, dữ liệu mẫu nằm ngoài lịch sử migration, nên bản triển khai thật và bản trình diễn dùng chung một lược đồ mà không lẫn dữ liệu.

Slide 14 — Lược đồ tổng thể: 20 thực thể `[bản đầy đủ]`

Hình: `hinh-3-2-erd-tong-the.png`

(82 từ · ~33 giây)

Nói:

Đây là lược đồ tổng thể, hai mươi thực thể ánh xạ tới hai mươi trong hai mươi mốt bảng. Em chia thành các nhóm: nhóm phòng và tiện nghi, nhóm đơn đặt phòng, nhóm thanh toán, nhóm nội dung trang chủ, và nhóm người dùng. Thiết kế theo hướng mỗi nhóm có một bảng gốc và các bảng phụ trỏ về, nên thêm một loại tiện nghi hay một kiểu nội dung mới là thêm dòng chứ không phải sửa bảng.

Slide 15 — Máy trạng thái đơn đặt phòng `[bản đầy đủ]`

Hình: `hinh-3-5-trang-thai-don.png`

(101 từ · ~40 giây)

Nói:

Đơn đặt phòng đi qua tám trạng thái. Em xin lưu ý hai đường chuyển ở phía dưới hình: đơn đã huỷ và đơn đã hết hạn vẫn quay lại được trạng thái chờ đối soát. Đó

là trường hợp tiền của khách về muộn — khách chuyển khoản chậm, đơn đã hết hạn giữ chỗ, nhưng tiền thì vẫn vào tài khoản thật. Nếu không có hai đường này thì tiền đã vào mà hệ thống coi như không có gì xảy ra. Em thiết kế sao cho không có nhánh nào để tiền của khách biến mất im lặng.

Slide 16 — Nhánh khó nhất: tiền về muộn `[bản đầy đủ]`

Hình: `hinh-3-8-tuan-tu-tien-ve-muon.png`

(113 từ · ~45 giây)

Nói:

Em muốn dừng ở một nhánh mà em nghĩ là khó nhất trong cả hệ thống. Khách chuyển khoản chậm, đơn đã hết hạn giữ chỗ mười lăm phút, nhưng tiền thì vẫn vào tài khoản thật. Nếu xử lý ngây thơ thì hệ thống sẽ bỏ qua webhook vì đơn không còn hiệu lực, và tiền của khách biến mất im lặng — khách mất tiền mà không có phòng. Sơ đồ này là cách em xử lý: webhook về muộn vẫn được ghi nhận, đơn quay lại trạng thái chờ đối soát, và chủ nhà thấy nó trên màn hình đối soát để quyết định hoàn tiền hay xếp phòng khác.

Slide 17 — Thanh toán: mã QR và webhook

Hình: `hinh-3-7-tuan-tu-thanh-toan.png`

(127 từ · ~51 giây)

Nói:

Luồng thanh toán như sau. Sau khi đặt, khách thấy mã QR theo chuẩn VietQR đã gắn sẵn số tiền và nội dung chuyển khoản, kèm đồng hồ đếm ngược mười lăm phút giữ chỗ. Khách quét và chuyển khoản; nhà cung cấp dịch vụ gọi webhook báo về cho hệ thống; hệ thống tự xác nhận đơn và gửi thư cho khách. Màn hình của khách tự chuyển khi tiền về, khách không phải bấm gì thêm. Điểm em muốn nhấn: trước khi cho một đồng nào ảnh hưởng tới dữ liệu, hệ thống kiểm năm lớp — khoá xác thực của webhook, đúng số tài khoản nhận, đúng chiều tiền vào, chống xử lý trùng một giao dịch, và cuối cùng là đúng số tiền.

Slide 18 — Thiết kế API: 82 thao tác trên 65 đường dẫn `[bản đầy đủ]`

Trên slide: Nhóm công khai: tìm phòng, đặt phòng, tra cứu đơn · Nhóm khách có tài khoản: xem lại đơn đã đặt · Nhóm quản trị: đơn, thanh toán, danh mục, nội dung, báo cáo · Nhóm webhook: chỉ nhà cung cấp thanh toán gọi, có khoá riêng

(79 từ · ~32 giây)

Nói:

> Giao diện lập trình ứng dụng có tám mươi hai thao tác trên sáu mươi lăm đường dẫn, chia bốn nhóm theo quyền. Nguyên tắc em đặt ra là quyền gắn vào nhóm đường dẫn chứ không gắn vào từng thao tác lẻ, vì gắn lẻ thì chỉ cần quên một chỗ là hở. Riêng nhóm webhook tách hẳn ra, dùng khoá riêng chứ không dùng token người dùng, vì người gọi nó là máy chứ không phải người.

Slide 19 — Mô hình bảo mật: mặc định từ chối tất cả

Trên slide: Cấu hình kết thúc bằng một quy tắc từ chối tất cả · Quên khai một đường dẫn mới thì nó bị CHẶN, không bị hở · Giới hạn tần suất theo 9 khoá: địa chỉ mạng, danh tính, mã đơn · 9 kịch bản tấn công thử trên hệ thống đang chạy — tất cả đều bị chặn

(133 từ · ~53 giây)

Nói:

> Về bảo mật, điểm thiết kế quan trọng nhất nằm ở dòng đầu: cấu hình phân quyền của em kết thúc bằng một quy tắc từ chối tất cả. Nghĩa là mặc định mọi đường dẫn đều bị chặn và phải khai tường minh mới mở ra. Nếu sau này em thêm một chức năng mới mà quên khai quyền, nó sẽ bị chặn chứ không bị hở — hệ thống hỏng theo hướng an toàn. Ngoài ra em giới hạn tần suất theo chín khoá khác nhau, và em thử chín kịch bản tấn công ngay trên hệ thống đang chạy: giả mạo địa chỉ, gọi endpoint quản trị bằng token khách, dùng token cũ sau khi đổi mật khẩu, gửi lại webhook, chèn thẻ kịch bản. Tất cả đều bị chặn.

Slide 20 — Giao diện khách: ba bước đặt phòng `[bản đầy đủ]`

Hình: `hinh-3-11-ba-buoc-dat-phong.png`

(93 từ · ~37 giây)

Nói:

Phía khách, luồng đặt phòng gọn trong ba bước: chọn ngày và số khách, chọn phòng, rồi xác nhận. Hai chi tiết nhỏ nhưng em cho là quan trọng. Thứ nhất, giá hiển thị là tổng tiền cả kỳ chứ không phải giá một đêm — đó mới là con số khách thực sự muốn biết trước khi quyết định. Thứ hai, những ngày đã kín phòng bị chặn sẵn ngay trong lịch, khách không chọn được; hệ thống không để khách điền xong hết thông tin rồi mới báo là hết phòng.

Slide 21 — Khu quản trị: 11 màn hình `[bản đầy đủ]`

Hình: `hinh-3-13-dashboard.png`

(71 từ · ~28 giây)

Nói:

Khu quản trị có mười một màn hình. Trang tổng quan hiện giá trị booking theo tháng, tỉ lệ lấp đầy, tỉ lệ huỷ, và số khoản tiền đang chờ đối soát. Em thiết kế khu này ưu tiên mật độ thông tin chứ không ưu tiên đẹp, vì chủ homestay mở nó hằng ngày và cần nhìn một lần là nắm được tình hình, không phải bấm qua nhiều lớp.

Slide 22 — Kết quả kiểm thử

Trên slide: 136 ca kiểm thử trong 19 lớp · Kiểm thử đa luồng với PostgreSQL thật, không giả lập · 11 lỗi thực tế đã tìm ra và khắc phục · Ma trận phân quyền: 30 tiền tố đường dẫn đều được khai

(144 từ · ~58 giây)

Nói:

Về kiểm chứng, hệ thống có 136 ca kiểm thử trong 19 lớp. Điểm em muốn nói rõ là kiểm thử tích hợp chạy trên PostgreSQL thật qua Testcontainers chứ không giả lập bằng cơ sở dữ liệu trong bộ nhớ — vì thứ cần kiểm chính là hành vi của PostgreSQL: ràng buộc loại trừ, kiểu khoảng, mã trạng thái SQL. Giả lập chúng là kiểm thử một thứ không tồn tại. Trong đó có ba ca kiểm thử đa luồng chứng minh trực tiếp rằng hệ thống không bán trùng phòng. Em cũng xin báo trung thực: lần

chạy đầy đủ gần nhất xanh 129 ca; bảy ca của lớp kiểm thử nút mô phỏng chuyển khoản bổ sung sau đó em mới kiểm chứng bằng cách gọi trực tiếp qua HTTP, chưa qua một lần chạy đầy đủ cả 136 ca.

Slide 23 — Mười một lỗi thực tế đã tìm ra và sửa `[bản đầy đủ]`

Hình: `hinh-4-1-do-tuong-phan.png`

(138 từ · ~55 giây)

Nói:

Em muốn nói một chút về mười một lỗi thực tế, vì em nghĩ chỗ này nói lên nhiều điều hơn là con số ca kiểm thử. Hình trên slide là một trong số đó: nút chính của trang có độ tương phản chỉ 2,27 trên 1, dưới xa ngưỡng 4,5 của chuẩn khả năng tiếp cận, do một quy tắc CSS nằm ngoài tầng. Sau khi sửa thì lên 7,69. Điều đáng nói là lỗi này biên dịch vẫn thành công, kiểm thử vẫn xanh, và chỉ lộ ra khi em chụp màn hình bằng trình duyệt thật. Bài học em rút ra là một màn hình chưa ai thao tác tới là một màn hình chưa được kiểm — và đó là lý do em bổ sung bước chụp ảnh tự động toàn bộ màn hình.

Slide 24 — Khả năng tiếp cận và triển khai

Trên slide: Quét 17 màn hình: 0 vi phạm WCAG 2.1 mức A và AA, 353 lần quy tắc đạt · Kiểm ở ba độ rộng màn hình, vùng chạm tối thiểu 44×44 điểm ảnh · docker compose up -d: bốn dịch vụ đều healthy

Hình: `hinh-4-2-quet-kha-nang-tiep-can.png`

(127 từ · ~51 giây)

Nói:

Bảng trên slide là kết quả quét thật chứ không phải con số em tự ghi: mỗi dòng là một màn hình, cột giữa là số vi phạm. Em quét tự động bằng axe-core trên mười bảy màn hình, bảy màn công khai và mười màn quản trị, và quét ngay trên bản đóng gói đang chạy — tức là đi qua đúng máy chủ web và đúng chính sách bảo mật nội dung của bản triển khai, chứ không phải chạy trên máy phát triển. Kết quả không còn lỗi vi phạm nào ở mức A và AA, trên tổng 353 lần quy tắc đạt. Về triển

khai, toàn hệ thống chạy được bằng ba lệnh, và cả bốn dịch vụ đều báo trạng thái khoẻ.

Slide 25 — Giới hạn đã biết

Trên slide: Chưa có bảng giá theo từng đêm — giá hiện là giá cố định theo loại phòng · Tầng giao diện chưa có bộ kiểm thử tự động thường trực · Hoàn tiền chưa tự động, chỉ ghi nhận để người xử lý

(99 từ · ~40 giây)

Nói:

Em xin nêu rõ giới hạn thay vì giấu đi. Ba giới hạn trên slide là những chỗ em biết hệ thống chưa làm được. Ngoài ra trong báo cáo em liệt kê đủ bảy giới hạn, và em cho rằng giới hạn quan trọng nhất là bản đóng gói hiện chưa có HTTPS, vì nó chạy trên máy cục bộ khi trình diễn — nếu đưa ra dùng thật thì đây là việc phải làm đầu tiên. Điểm chung của những giới hạn này là chúng đều là chỗ mở rộng được, không phải chỗ thiết kế sai.

Slide 26 — Hướng phát triển

Trên slide: Bảng giá theo mùa và theo ngày trong tuần · Ứng dụng di động — dùng lại nguyên giao diện lập trình ứng dụng · Đồng bộ lịch hai chiều với sàn trung gian

(95 từ · ~38 giây)

Nói:

Ba hướng phát triển tiếp theo. Bảng giá theo mùa là thứ cần nhất về nghiệp vụ, vì giá cuối tuần và giá lễ khác hẳn ngày thường. Về ứng dụng di động, chính vì em tách rời giao diện và máy chủ ngay từ đầu nên ứng dụng di động dùng lại được nguyên giao diện lập trình ứng dụng hiện có, không phải viết lại tầng máy chủ. Hướng thứ ba là đồng bộ lịch hai chiều với sàn trung gian, để chủ nhà vẫn bán trên sàn mà không lo trùng lịch.

Slide 27 — Liên kết

Trên slide: Video thuyết trình: «DÁN LIÊN KẾT GOOGLE DRIVE VÀO ĐÂY» · Mã nguồn và toàn bộ tài liệu: · github.com/nhquan403/cn-dx23tt11-nguyenhongquan-homestay-tvu · Bản trình diễn hệ thống 10 phút: thesis/abs/ trong kho mã nguồn

(51 từ · ~20 giây)

Nói:

> Slide này để thầy cô mở lại bất cứ lúc nào: video thuyết trình, kho mã nguồn và bản trình diễn hệ thống dài mười phút đi hết toàn bộ quy trình. Kho mã nguồn em để công khai nên thầy cô mở được ngay, không cần xin quyền truy cập.

Slide 28 — Cảm ơn

(29 từ · ~12 giây)

Nói:

> Phần trình bày của em đến đây là hết. Em xin cảm ơn thầy cô đã lắng nghe, và em sẵn sàng trả lời câu hỏi của thầy cô.

DỰ KIẾN CÂU HỎI CỦA HỘI ĐỒNG

Phần này **không lên slide**. Mỗi câu dưới đây nhắm vào một quyết định kỹ thuật mà đồ án đã chọn một trong nhiều đường, hoặc một giới hạn đã ghi trong báo cáo — đó là hai chỗ hội đồng hay hỏi nhất.

Đọc để **nắm ý**, đừng học thuộc. Trả lời bằng lời của mình thì nghe thật hơn, và nếu bị hỏi vặn tiếp thì vẫn xoay được.

1. Vì sao chọn PostgreSQL mà không phải MySQL?

Vì một tính năng rất cụ thể: ràng buộc loại trừ EXCLUDE kết hợp kiểu dữ liệu khoảng daterange và chỉ số GiST. Đó chính là thứ cho phép em chống đặt trùng phòng ở tầng lưu trữ. MySQL không có cả kiểu khoảng lẫn ràng buộc loại trừ, nên nếu dùng MySQL em sẽ buộc phải quay lại kiểm tra ở tầng ứng dụng — tức là quay lại đúng cái khe hở mà đề tài muốn bịt. Em chọn theo bài toán chứ không chọn theo cảm tính.

2. Ràng buộc loại trừ như vậy có làm chậm hệ thống không?

Có chi phí, vì mỗi lần ghi phải kiểm chỉ số GiST. Nhưng đổi lại là tính đúng đắn, và ở quy mô một homestay thì số đơn mỗi ngày rất nhỏ so với khả năng của PostgreSQL. Em cho rằng đánh đổi này đáng: một hệ thống nhanh mà bán trùng phòng thì không dùng được, còn chậm hơn vài mili giây thì không ai nhận ra.

3. Khi bị cơ sở dữ liệu bác thì khách thứ hai thấy gì?

Khách thứ hai không thấy lỗi. Hệ thống bắt đúng mã lỗi 23P01, rồi tự thử phòng trống tiếp theo trong cùng loại phòng, trong một giao dịch mới. Chỉ khi loại phòng đó thật sự hết sạch thì khách mới nhận thông báo hết phòng. Nghĩa là ràng buộc ở tầng dữ liệu không biến thành trải nghiệm xấu ở tầng giao diện.

4. Nếu webhook không tới thì sao? Tiền khách đã chuyển rồi.

Đây là trường hợp em có xử lý riêng. Đơn sẽ hết hạn giữ chỗ sau mười lăm phút, nhưng máy trạng thái cho phép đơn đã hết hạn hoặc đã huỷ quay lại trạng thái chờ đối soát khi webhook về muộn. Ngoài ra khu quản trị có màn hình đối soát để chủ

nhà xử lý tay những khoản thiếu tiền hoặc thừa tiền. Nguyên tắc em đặt ra là không có nhánh nào để tiền của khách biến mất im lặng.

5. Làm sao chống việc gửi lại cùng một webhook nhiều lần?

Em chống trùng theo cặp nhà cung cấp và mã giao dịch bên ngoài. Lần đầu nhận thì ghi nhận cặp đó; những lần sau cùng cặp đó sẽ bị bỏ qua, không cộng tiền thêm lần nữa. Em có một kịch bản tấn công thử đúng việc này — gửi lại cùng một webhook — và nó bị chặn.

6. Màn thanh toán tự cập nhật — em dùng WebSocket à?

Không, em dùng cách đơn giản hơn: màn hình hỏi lại trạng thái đơn vài giây một lần. Với bài toán này thì đủ, vì chỉ có một màn hình chờ trong mười lăm phút và độ trễ vài giây không ảnh hưởng gì. WebSocket sẽ thêm một kênh kết nối phải duy trì và phải xử lý khi rớt mạng, trong khi lợi ích gần như bằng không ở quy mô này.

7. Em đã chạy đủ bộ kiểm thử chưa?

Em xin trả lời thẳng: chưa đủ hoàn toàn. Lần chạy đầy đủ gần nhất xanh 129 ca. Sau đó em bổ sung một lớp kiểm thử bảy ca cho nút mô phỏng chuyển khoản, và bảy ca này em mới kiểm chứng bằng cách gọi trực tiếp qua HTTP trên hệ thống đang chạy, chưa qua một lần chạy đầy đủ cả 136 ca. Em có ghi rõ điều này trong báo cáo ở mục 4.6.3 chứ không giấu.

8. Hệ thống chống XSS và SQL injection thế nào?

SQL injection thì em dùng truy vấn tham số hoá qua tầng ánh xạ đối tượng quan hệ, không nối chuỗi SQL. XSS thì nội dung do người quản trị soạn được lọc HTML trước khi lưu, và em có kịch bản tấn công thử chèn thẻ kịch bản vào nội dung — nó bị chặn. Tổng cộng em thử chín kịch bản tấn công trên hệ thống đang chạy, tất cả đều bị chặn, chi tiết ở Phụ lục M.

9. Làm sao chắc là không sót endpoint nào quên phân quyền?

Cấu hình bảo mật của em kết thúc bằng một quy tắc từ chối tất cả. Nghĩa là mặc định mọi đường dẫn đều bị chặn, và phải khai tường minh mới mở. Nếu em thêm

một endpoint mới mà quên khai thì nó bị chặn chứ không bị hở — hỏng theo hướng an toàn. Em cũng có một ma trận phân quyền kiểm 30 tiền tố đường dẫn đều đã được khai.

10. Vì sao bản đóng gói chưa có HTTPS?

Vì bản đóng gói hiện chạy trên máy cục bộ để trình diễn, chưa có tên miền nên chưa xin được chứng chỉ. Em ghi đây là giới hạn số bảy trong báo cáo và coi nó là giới hạn quan trọng nhất: nếu đưa hệ thống ra dùng thật thì phải bổ sung chứng chỉ đầu tiên, vì hiện mật khẩu và token đi qua mạng ở dạng rõ.

11. Hệ thống chịu được bao nhiêu người dùng cùng lúc?

Em chưa đo tải nên em không dám đưa con số. Thứ em đo được và dám khẳng định là tính đúng đắn khi có tranh chấp: ba ca kiểm thử đa luồng cho thấy dù nhiều luồng cùng đặt một phòng thì chỉ một đơn được ghi. Về tải, em có biết một giới hạn cụ thể: bộ đếm giới hạn tần suất hiện lưu trong bộ nhớ tiến trình, nên chỉ đúng khi chạy một bản ứng dụng; muốn chạy nhiều bản thì phải chuyển sang bộ nhớ đệm dùng chung.

12. Giá phòng cố định thì mùa cao điểm tính sao?

Đúng là hiện giá cố định theo loại phòng, và em ghi đây là giới hạn. Về mặt thiết kế thì chỗ mở rộng đã có sẵn: giá đang là thuộc tính của loại phòng, thêm bảng giá theo khoảng ngày là thêm một bảng tra cứu rồi đổi chỗ tính tiền, không phải sửa lõi chống trùng lịch. Em xếp nó là hướng phát triển số một vì nó cần nhất về nghiệp vụ.

13. Vì sao không làm ứng dụng di động?

Vì phạm vi đồ án cơ sở ngành, và vì trang web đáp ứng đã giải quyết được nhu cầu trước mắt — em có kiểm giao diện ở ba độ rộng màn hình, vùng chạm tối thiểu 44 nhân 44 điểm ảnh. Nhưng kiến trúc em chọn là tách rời, nên khi làm ứng dụng di động thì dùng lại được nguyên giao diện lập trình ứng dụng hiện có, không phải viết lại tầng máy chủ.

14. Chưa có tài khoản SePay thật thì em trình diễn thanh toán kiểu gì?

Em bổ sung một nút mô phỏng chuyển khoản, chỉ bật ở cấu hình trình diễn và mặc định tắt trong mã nguồn. Nút đó dựng một thông báo chuyển khoản giống hệt thông báo thật của nhà cung cấp rồi đẩy qua đúng đường xử lý thật — nghĩa là vẫn đi qua đủ năm lớp kiểm. Nên luồng chạy y như khi tiền về thật, chỉ khác ở chỗ ai là người phát ra thông báo.

15. Dữ liệu mẫu trong bản trình diễn có phải tự bịa không?

Dữ liệu là sinh ra để trình diễn, em không giấu điều đó. Nhưng nó được sinh có chủ đích để lộ ra các trường hợp khó chứ không phải cho đẹp: có cuối tuần cháy phòng để thấy lịch chặn sẵn, có khoảng đóng phòng đang áp dụng và đã qua, có khoản thiếu tiền và thừa tiền ở màn đối soát, có cả một thư gửi thất bại. Mọi ngày trong dữ liệu là ngày tương đối so với hôm nay, nên bộ dữ liệu không cũ đi theo thời gian.

Nếu gặp câu không biết: nói thẳng là em chưa tìm hiểu chỗ đó, rồi nêu cái gần nhất mình có căn cứ. Hội đồng trừ điểm vì trả lời vòng vo nhiều hơn là vì một câu không biết.